Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 00 (2025) 000–000

Procedia Computer Science

www.elsevier.com/locate/procedia

12th. Federation of European Simulation Societies Conference, (Eurosim 2026)

A Task-Oriented Executable Modeling Framework for Smart Chemical Laboratory

Tao Yu[a], Chun Zhao[a,*]

[a]*Beijing Information Science & Technology University, Qinghe Xiaoying East Road 12, Haidian District, Beijing 100192, China*

Abstract

Smart chemical laboratory automation increasingly demands an explicit representation of experimental task execution that goes beyond device command orchestration. Existing approaches—script-driven automation, workflow orchestration, and model-based system description—treat device invocation, process organization, and asset-level representation as separate concerns, and none provides a unified task-level model in which device capabilities, runtime states, data dependencies, and execution constraints are organized as coordinated elements of a task-specific system representation. This paper proposes an executable modeling framework in which a workflow is defined as a task-specific execution slice of the laboratory system. Within this slice, each workflow node is formalized as a process-level mapping of a device capability, carrying explicit state, data, and signal semantics rather than serving as a simple command wrapper. The framework is organized across three coordinated layers: a static resource layer that encapsulates physical devices as executable object models with capability definitions and state machines; a dynamic workflow layer that organizes device capabilities and their temporal, control, and data relationships; and a constraint layer that monitors laboratory-wide and task-specific boundaries during execution. The framework is instantiated in a prototype system and demonstrated through a reactor-heating task. The case study shows that device abstraction, workflow execution, and constraint supervision can be integrated within a single executable model, making the dynamic evolution of the laboratory system explicit and inspectable at the model level.

Keywords: Experimental task execution; Execution semantics; Workflow modeling; Workflow-based execution; Smart laboratory; Laboratory automation

1. Introduction

The development of smart chemical laboratories is driving laboratory automation toward integrated task execution in which devices, processes, and constraints are coordinated within a unified operational context [1]. Representing such execution requires more than ordering device operations: it must also capture the organization of task-relevant

* Corresponding author. Tel.: +86-13311296283.
E-mail address: zhaochun@bistu.edu.cn (Chun Zhao).

resources, the evolution of system states, the propagation of process data, and the maintenance of execution boundaries [2].

Many existing laboratory automation systems implement experimental tasks through scripts and application programming interfaces [3]. These approaches are effective for device integration and runtime control, but device state transitions, data propagation, and constraint handling are implemented across interface definitions and runtime code rather than organized as explicit model-level constructs [2]. Workflow-based approaches provide structured representations of experimental procedures, capturing process steps, control flow, and execution dependencies [4]; however, workflow nodes are treated as process actions rather than as formal representations of device capabilities, and therefore do not carry device state, data dependency, or constraint semantics [5]. Model-based and digital-twin-oriented approaches provide structured descriptions of laboratory resources and behavioral relationships [6, 7, 8], but do not specify how task-relevant capabilities should be selected from resource models and assembled—together with their runtime relationships and constraints—into an executable representation for a specific task. The remaining gap is therefore not the absence of automation, workflow descriptions, or system models, but the lack of a task-level model that organizes device capabilities, runtime states, data dependencies, and execution constraints as coordinated elements of a task-specific system representation.

To address this gap, this paper proposes an executable modeling framework that defines a workflow as a task-specific execution slice of the laboratory system. Within this slice, each workflow node is formalized as a process-level mapping of a device capability, carrying explicit state, data, and signal semantics, rather than serving as a simple command wrapper. The framework organizes experimental task execution across three coordinated layers: a static resource layer that provides executable device models with capability definitions and state machines; a dynamic workflow layer that organizes device capabilities and the temporal, control, and data relationships among them; and a constraint layer that supervises laboratory-wide and task-specific boundaries during execution. Together, these layers make the organizational structure and dynamic evolution of the laboratory system explicit at the model level.

The main contributions of this paper are as follows:

- A task-oriented executable modeling framework is proposed in which a workflow is defined as a task-specific execution slice of the laboratory system. Unlike existing workflow models that treat nodes as process actions, this framework formalizes workflow nodes as process-level mappings of device capabilities with explicit state, data, and signal semantics, and integrates constraint supervision within a unified model.

- The framework is instantiated in a prototype system and demonstrated through a reactor-heating case study, showing that device abstraction, workflow execution, and constraint supervision can be organized as coordinated model elements, with execution semantics made explicit and inspectable at the model level rather than distributed across interface definitions and runtime code.

The remainder of this paper is organized as follows: Section 2 reviews related work; Section 3 presents the proposed framework; Section 4 reports the case study and discussion; Section 5 concludes.

2. Related Work

Existing literature on smart chemical laboratory systems can be broadly grouped into three lines of research according to how experimental task execution is represented: script- and API-oriented laboratory automation, workflow-based process orchestration, and model-based representation of laboratory systems [9, 10].

The first line of research focuses on script-driven automation and device APIs [10]. In these approaches, device heterogeneity is commonly handled through interface encapsulation, while experimental tasks are implemented using scripts or templates [3]. Such methods are effective for device integration, command dispatch, and engineering implementation [9, 11, 12, 13]. However, device states, data propagation, and constraint handling are typically distributed across scripts, adapters, and runtime control code rather than organized as a unified task-level model.

The second line of research uses workflows or process models to represent experimental procedures, providing structured descriptions of process steps, control flow, and execution dependencies [4, 14]. In conventional workflow representations, nodes are generally interpreted as actions, services, or processing steps, and the workflow primarily specifies how experimental steps are organized and how execution progresses [15]. Such representations are suitable for process orchestration, but they do not explicitly connect workflow nodes with device capability models, device state evolution, data relationships, and execution constraints as coordinated elements of a task-specific laboratory

representation. Workflow models therefore describe how a task proceeds, but do not represent how the laboratory system is organized and evolves during task execution.

The third line of research includes model-based, MBSE-oriented, and digital-twin-based representations of laboratory systems [6, 7, 8, 16]. These approaches provide structured descriptions of laboratory resources, system structures, capabilities, behavioral relationships, and physical–digital interactions, offering a systematic basis for modeling laboratory assets beyond implementation-specific automation logic. However, these models are generally constructed from a system-level or asset-level perspective and do not define how task-relevant capabilities should be selected from existing resource models and how their runtime states, data and signal relationships, and execution constraints should be organized into an executable representation for a specific experimental task [2].

Collectively, these approaches address device invocation, process orchestration, and system-level resource modeling as separate concerns. However, none provides a task-level model in which device capabilities, runtime states, data dependencies, and execution constraints are organized as coordinated elements of a task-specific executable representation.

3. Methodology

Experimental task execution is not merely the sequential invocation of device commands, but the overall dynamic evolution of the laboratory system under a specific task context [17]. Therefore, the explicit modeling of experimental task execution must cover three core elements, resources that persist and evolve during operation, workflow organization tailored to specific tasks, and constraints that ensure all execution activities remain within permissible boundaries [18]. Based on this understanding, this paper constructs a unified modeling framework that characterizes experimental task execution from three perspectives, namely resource representation, workflow abstraction, and constraint supervision. The following subsections present the system abstraction and the formal definitions of the proposed framework.

3.1. System Abstraction

The laboratory system is abstracted as a four-component tuple:

$$\mathcal{L} = \langle \mathcal{R}, \mathcal{F}, C, \Pi \rangle \tag{1}$$

where $\mathcal{R}$ is the static resource layer modeling device assets and their connections, $\mathcal{F}$ is the dynamic workflow layer organizing task-specific execution, C is the constraint layer supervising operational boundaries, and Π is the protocol specification defining signal formats and interaction conventions shared across the three layers. The following subsections detail each layer.

3.2. Static Resource Layer

The static resource layer $\mathcal{R}$ provides a unified executable semantic basis for the other layers by modeling device assets and their connections.

The core modeling principle of this layer is to separate device capabilities from device states. Each physical device is abstracted as a device object model composed of a capability model and a state machine. The capability model describes the static information and capability semantics of the device, while the state machine describes execution interactions and state transitions.

Formally, the static resource layer is defined as

$$\mathcal{R} = \langle V_D, G_C \rangle \tag{2}$$

where $V_D = \{D_1, D_2, \ldots, D_n\}$ is the set of device object models, and G_C is the organizational structure of device resources.

Each device object model $D \in V_D$ is defined as $D = \langle Cap_D, SM_D \rangle$, where Cap_D is the device capability model and SM_D is the device state machine.

The device capability model is defined as

$$Cap_D = \langle Attr,\ Ops,\ Adpt,\ Port_D,\ Constr_D \rangle \tag{3}$$

where $Attr$ denotes the device attribute space, including continuous and discrete attributes; Ops denotes the set of operational capabilities exposed by the device; $Adpt$ denotes the adapter contract, which specifies command templates, interaction events, and feedback signals between the adapter and the device object model; $Port_D$ denotes the set of attribute-data interaction ports; and $Constr_D$ denotes the device-intrinsic constraints, namely the set of device-level operational constraints that remain active independently of the task context.

Each element in $Constr_D$ specifies a device-local boundary condition of the form $\langle att,\ op,\ val \rangle$, where att is a device attribute, op is a comparison operator, and val is a constraint boundary or value range. During operation, the device state machine continuously monitors device attributes according to $Constr_D$. Once a constraint violation is detected, the state machine immediately enters the corresponding abnormal state.

The adapter contract $Adpt$ is logically associated with an adapter component that controls interactions with the physical device and reports execution status to the device object model. The adapter registers its command templates and event signals with the model, while the capability model provides a unified semantic view of this information.

In this design, Cap_D serves as the static semantic basis of the device. It does not actively execute operations but provides a structured description of device attributes, executable capabilities, and device-intrinsic constraints. Device attributes can be obtained directly from $Attr$ without being relayed through the state machine.

The dynamic behavior of the device is governed by the device state machine, defined as

$$SM_D = \langle St,\ Ifc,\ Act,\ Trans,\ s_0 \rangle \tag{4}$$

where $St = St^{cmd} \times St^{op}$ denotes a composite state space consisting of command lifecycle states and device operation states; Ifc denotes the interface set; Act denotes the set of internal state-machine actions; $Trans$ denotes the set of state-transition rules; and s_0 denotes the initial state.

Dividing the state space into command lifecycle states and device operation states reflects the dual nature of device execution. The command lifecycle state describes the execution progress of an operation command, while the device operation state reflects the actual physical operating condition of the device.

The interface set Ifc defines the interaction boundary of the state machine. Specifically, ifc^{in}_{cmd} receives device control commands, ifc^{in}_{adp} receives feedback events from the adapter, ifc^{out}_{adp} sends operation commands to the adapter, and ifc^{out}_{stat} publishes device states externally. Interfaces serve only as communication endpoints and do not carry execution logic.

The internal action set Act contains a minimal set of action primitives, such as sending control commands to the adapter or emitting signals through interfaces. These actions are triggered during state transitions or upon state entry.

Each state-transition rule $tr \in Trans$ is defined as

$$tr = \langle s_{src},\ ifc,\ \sigma,\ s_{tgt},\ Act_t \rangle \tag{5}$$

where s_{src} and s_{tgt} denote the source and target states, respectively; ifc denotes the interface that receives the triggering signal; σ denotes the triggering signal; and $Act_t \subseteq Act$ denotes the set of actions executed during the transition. State transitions are jointly determined by the current state, the input interface, and the received signal. Each state may also define corresponding entry actions.

The system connection graph is defined as

$$G_C = \langle V_C,\ E_C \rangle \tag{6}$$

where $V_C = \bigcup_{D \in V_D} Port_D$ denotes the global set of device ports in the laboratory, and $E_C \subseteq V_C \times V_C$ denotes the set of port-to-port connections. This graph describes attribute-level data exchange relationships among device assets.

During experimental task execution, workflow nodes interact with device object models through state-machine interfaces. After receiving a control signal, SM_D performs a state transition, sends operation commands to the adapter, and subsequently receives feedback events and publishes device states through its interfaces. Concurrently, SM_D monitors the device according to $Constr_D$; upon detecting a violation, it immediately enters the corresponding abnormal state. Through this mechanism, physical devices are transformed into controllable, state-aware, and executable digital object models.

3.3. Dynamic Workflow Layer

The dynamic workflow layer $\mathcal{F}$ describes the device capabilities organized during task execution and their data, control, and temporal relationships.

Formally, the dynamic workflow layer is defined as

$$\mathcal{F} = \langle V_F, E_I, E_P \rangle \tag{7}$$

where V_F denotes the set of workflow nodes, E_I denotes the set of interface connections, and E_P denotes the set of port connections.

The topological structure of the workflow is jointly determined by E_P and E_I. The port connection set E_P represents persistent data-dependency channels between nodes. The interface connection set E_I specifies the routing relationships for signal interaction, including interaction channels between nodes within the workflow layer and cross-layer interaction channels between nodes and device state machines. The control topology between nodes can be regarded as a subset of the interface connections.

Within this topological structure, each node $n \in V_F$ is defined as

$$n = \langle Map, Var, St, Trans, Ifc, Port, Act \rangle \tag{8}$$

where Map is the node capability mapping, Var is the internal variable space, St is the set of node lifecycle states, $Trans$ is the lifecycle transition rules, Ifc is the interface set, $Port$ is the port set, and Act is the set of internal node actions.

The node capability mapping Map denotes the semantic function of a node in the workflow. For an execution node, $Map = \langle D, op \rangle$, where D is a device object model and $op \in Ops_D$ is one operational capability of that device. For a functional node, Map denotes control functions such as conditional branching, synchronization, or waiting.

The port set $Port$ represents the endpoints through which a node exchanges data with other nodes via E_P connections. The internal variable space Var maintains the local context during node execution, including user-defined variables, port-bound data variables, and attribute data mapped from the device capability model.

The lifecycle state set St describes the process-level execution state of a node, typically including pending, running, completed, and failed. This lifecycle is affected by the state returned from SM_D, but the two are not equivalent. $Trans \subseteq St \times St$ defines the allowed lifecycle transition rules. It does not execute actions by itself but constrains valid transitions between lifecycle states.

The interface set Ifc serves as the endpoints for signal interaction. Each interface $ifc \in Ifc$ is defined as

$$ifc = \langle name, dir, type, Sig, Trig \rangle \tag{9}$$

where dir denotes the direction, $type$ denotes the interface category, Sig defines the set of signals allowed to pass through the interface, and $Trig$ denotes the set of triggers bound to the interface. Each trigger is defined as $trig = \langle cond, act \rangle$, where $cond$ is a triggering predicate defined over the node context, and $act \in Act$ is the node action executed when the condition is satisfied. A node action may update the lifecycle state, modify internal variables, or emit signals through interfaces.

During experimental task execution, after an input interface of a node receives a signal, the triggers bound to that interface check whether their predicates are satisfied. If a condition is satisfied, the corresponding action is invoked, updating the node context and potentially requesting a lifecycle state change. This change must satisfy the transition rules specified by $Trans$. After the context is updated, triggers on other interfaces continue to evaluate their conditions; if satisfied, the corresponding actions are executed, such as sending a signal through an output interface. Meanwhile, data between nodes remain continuously available through port connections while the connections are valid. Through this mechanism, the workflow coordinates device interactions and represents the state evolution of the laboratory system during task execution.

3.4. Constraint Layer

The constraint layer C describes laboratory-wide and task-specific boundary constraints and monitors whether the laboratory state remains within admissible boundaries during execution.

Formally, the constraint layer is defined as

$$C = \langle BC, Obs, VH, TC \rangle \tag{10}$$

where BC denotes the set of laboratory-wide baseline constraints, Obs denotes the observable object space, VH denotes the set of violation-handling actions, and TC denotes the set of task constraint packages.

The observable object space is defined as $Obs = Obs_R \cup Obs_F$, where Obs_R denotes observable objects provided by the static resource layer, including device attributes, command lifecycle states, and device operation states; and Obs_F denotes observable objects provided by the dynamic workflow layer, including node lifecycle states and signal variables.

The set VH includes violation-handling actions such as warning, alarm, stop, and abort. These actions are part of the constraint rules and distinguish different types of violation consequences.

BC denotes the set of laboratory-wide baseline constraints that remain continuously active and constrain the operating state of the entire laboratory over time. Each constraint $c \in BC$ is defined as

$$c = \langle obj, op, val, hdl \rangle \tag{11}$$

where $obj \in Obs$ denotes the observed object, op denotes a comparison operator, val denotes a constraint boundary or value range, and $hdl \in VH$ denotes the handling action executed upon violation. For example, $\langle temp, \leq, 200, stop \rangle$ indicates that the observed temperature must not exceed 200; if violated, the action $stop$ is executed.

For a task instance τ, the task constraint package is defined as $TC_\tau = \langle G_\tau, BC_\tau \rangle$, where G_τ denotes the task goal set and BC_τ denotes the boundary constraint set specific to that task. BC remains active regardless of whether a task is running, whereas TC_τ is introduced only during the execution of a specific task.

The constraint layer distinguishes two levels of operational boundaries. Baseline constraints in BC govern the laboratory session as a whole; task constraints in TC_τ apply only to the active task phase. Device-intrinsic constraints defined in $Constr_D$ are enforced locally by SM_D: when a device attribute violates a boundary in $Constr_D$, the state machine immediately enters the corresponding abnormal state, which is then observable via Obs_R. If a matching constraint exists in BC or BC_τ, the constraint layer additionally executes the corresponding system-level handling action. The two mechanisms therefore operate in a complementary manner: $Constr_D$ provides immediate local enforcement, while the constraint layer performs continuous global and task-level supervision over the observable space.

4. Results and Discussion

4.1. Case Study

To illustrate how the proposed framework is instantiated and implemented in practical scenarios, this paper adopts a representative reactor heating task as a case study. This case demonstrates three core dimensions of the framework: device abstraction at the resource layer, device capability mapping at the workflow layer, and runtime supervision at the constraint layer. The case provides a preliminary demonstration of the framework's structural instantiability and runtime executability rather than a quantitative performance benchmark.

Figure 1 shows the instantiated architecture and its implementation across the three layers. The physical reactor is instantiated as an object model with capability definitions and a state machine; its heating capability is mapped to a workflow node forming part of the execution slice; and the constraint layer monitors observable objects from both layers. With this architecture, the dynamic evolution of experimental task execution is explicitly represented.

Figure 2 shows the prototype interface. The upper part presents the state-machine configuration of the reactor model, where states, triggering conditions, and transition actions are defined. The lower part presents the workflow modeling interface, where node variables, ports, and interfaces can be configured. In this example, the reactor temperature attribute is mapped to the internal variable space of the workflow node and connected to other nodes through data ports (yellow dashed line).

Figure 3 shows the cross-layer execution sequence. The heating node sends an execution signal to the device state machine, which triggers a state transition and sends a heating command to the adapter. The adapter translates the command into physical device protocols and controls the reactor, returning feedback events such as acknowledgment and status updates during execution. These events drive the state evolution of the device state machine, and the updated

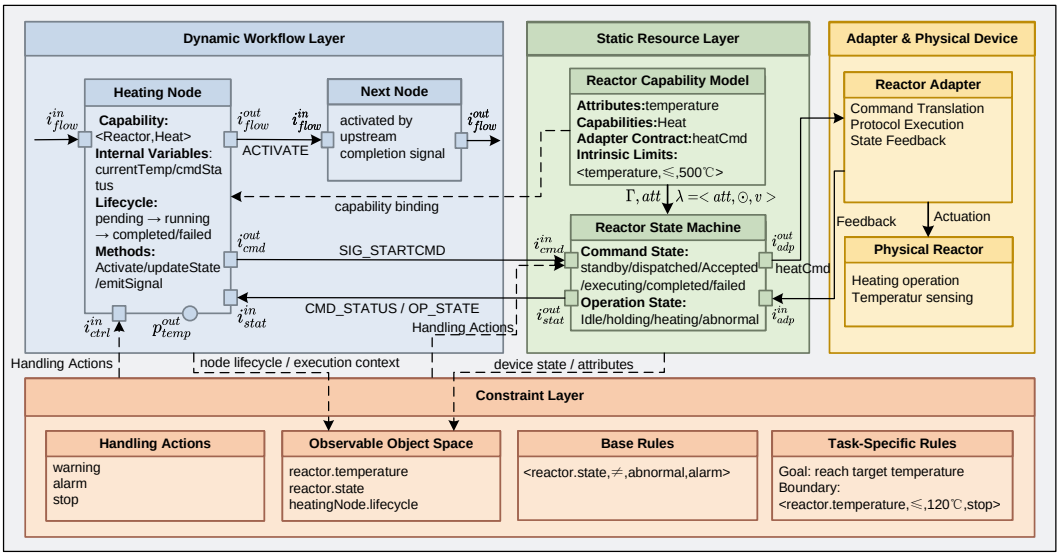

Fig. 1. Instantiated architecture of the proposed modeling framework for the reactor heating task.

state is fed back to the workflow node, which then updates its lifecycle state and signals the downstream node. An exception branch illustrates two constraint scenarios: the constraint layer directly detects that the reactor temperature exceeds a task boundary in the observable object space, or the device state machine detects a temperature exceeding the device-intrinsic limit and enters an abnormal state, which is subsequently observed by the constraint layer. Both cases trigger handling actions such as issuing warnings or sending stop signals to relevant nodes.

The case study demonstrates that the proposed framework can be instantiated and executed in a concrete laboratory scenario. At the resource layer, the physical reactor is abstracted as an object model with formally specified capability definitions and a state machine, without embedding execution logic in scripts or adapter code. At the workflow layer, the device capability is mapped to a workflow node through the capability mapping mechanism, allowing the workflow to reference the device model directly rather than re-implementing its behavior. At the constraint layer, supervision operates at both the device-intrinsic and task-specific levels, with each level triggering distinct handling actions as intended. These results show that device abstraction, workflow execution, and constraint supervision can be organized as coordinated model elements, making the dynamic evolution of the laboratory system explicit and inspectable at the model level.

4.2. Discussion

The proposed framework organizes device capabilities, workflow structures, runtime states, and constraint rules as coordinated model elements rather than distributing them across scripts, adapters, and runtime control code. This organization improves the observability, configurability, and semantic transparency of experimental task execution.

The applicability of the framework depends on the extent to which laboratory resources and experimental procedures can be represented using device object models, workflow nodes, and constraint rules. For laboratory scenarios involving heterogeneous device types, multi-device coordination, data- and state-dependent execution, and task-specific operational boundaries, the three-layer structure provides a natural organizational basis. Device models can be defined independently of specific task contexts and referenced across different experiments; workflow nodes can be configured to reference existing device capabilities without requiring additional scripting; and constraint rules can be specified at the laboratory-wide and task-specific levels without modifying device models or workflow definitions. These properties make the framework applicable to a range of experimental task types that require explicit coordination of device resources, process data, and execution boundaries.

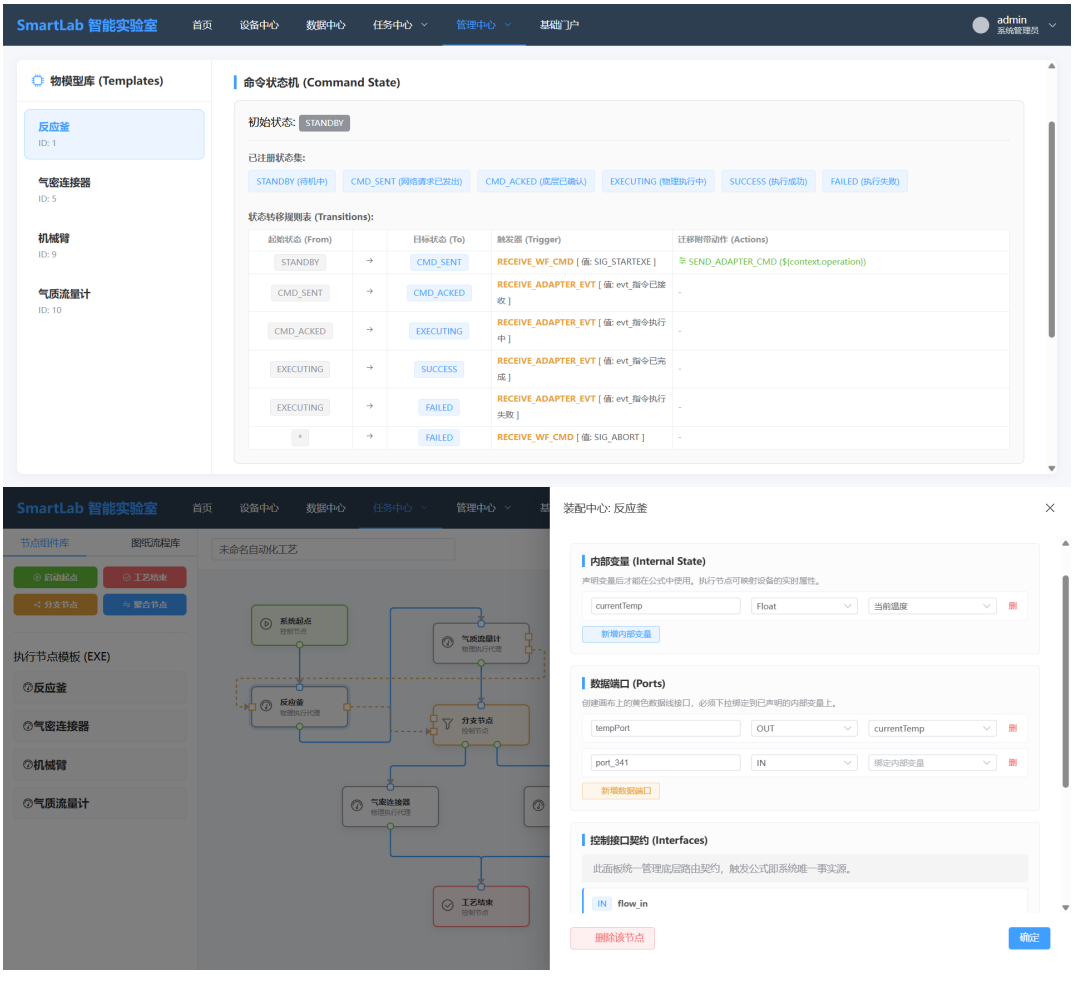

Fig. 2. Prototype interface of the proposed modeling framework for configuring device state machines and workflow nodes.

Regarding scalability, the separation of device models, workflow structures, and constraint supervision into independent but coordinated layers provides a structural basis for managing increased complexity. As the number of devices grows, each new device can be integrated by defining its Cap_D and SM_D without altering existing workflow or constraint definitions. As workflow complexity increases, new nodes can be assembled from existing device models through capability mapping, and constraint packages can be added per task without structural changes to the resource or workflow layers. As the number of constraints increases, the observable space Obs and constraint sets BC and BC_τ can be extended independently. However, as the number of devices, workflow nodes, and concurrent constraint checks grows, event routing, resource binding, and model consistency maintenance may introduce additional complexity that requires further investigation.

The current validation demonstrates the instantiability and preliminary executability of the framework but does not yet provide systematic evidence for large-scale, multi-device, highly concurrent, or long-duration scenarios. Further validation is required for tasks involving multiple devices, exception branches, simulation-supported execution, and cross-task model reuse.

Future quantitative evaluation can be conducted across three metrics: representational coverage, which evaluates the extent to which the framework can express representative laboratory task requirements; model reuse rate, which evaluates whether existing device models, workflow fragments, and constraint rules can be reused for new task con-

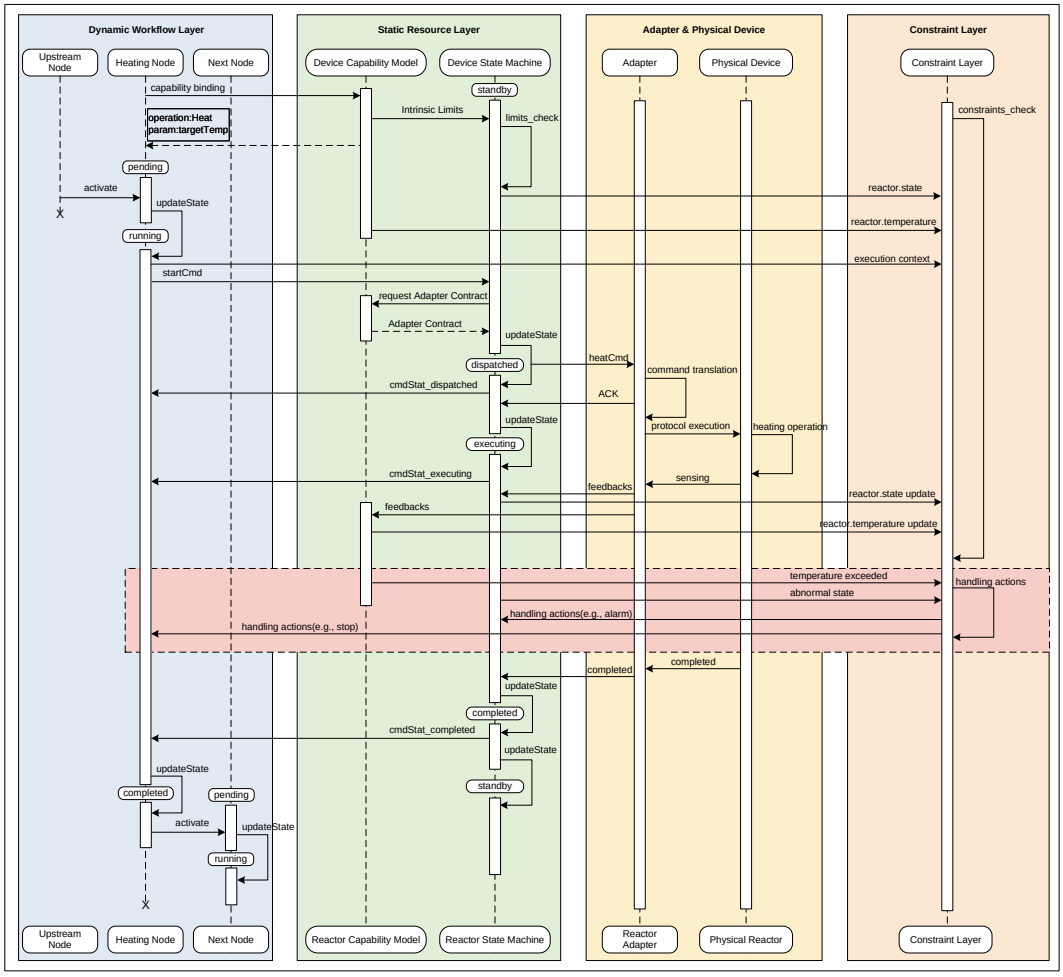

Fig. 3. Cross-layer execution sequence of the reactor heating task.

struction without modifying their structural definitions; and model change-impact scope, which measures the scope of affected model elements when task requirements or laboratory resources change. These metrics can be complemented by a structured feature-level comparison against representative existing approaches.

5. Conclusions

This paper proposes a task-oriented executable modeling framework that represents the laboratory system as a four-component structure comprising a static resource layer, a dynamic workflow layer, a constraint layer, and a protocol specification. The framework integrates device capabilities, runtime states, data dependencies, and execution constraints within a task-specific executable representation.

In the resource layer, each device is represented as a device object model that formally separates capability definitions from behavioral state transitions, providing an executable semantic basis that the workflow layer can reference directly. The workflow layer represents experimental procedures as task-specific execution slices, in which workflow nodes serve as process-level mappings of device capabilities with explicit lifecycle states, signal interfaces, and data ports. The constraint layer supervises execution at two complementary levels: device-intrinsic constraints, enforced locally by each device state machine, and task-specific constraints, monitored globally through a unified observable

object space. The case study demonstrates that this structure can be instantiated in a concrete laboratory scenario: a physical device can be abstracted as an executable object model with formal capability and state definitions, device capabilities can be mapped to workflow nodes without additional scripting, and constraint supervision functions at both levels within the same model.

The current work concentrates on the formal proposal and illustrative instantiation of the framework; systematic validation in complex, multi-device, and long-duration experimental scenarios remains to be conducted. Future work will focus on extending the prototype system, supporting simulation-based execution, conducting systematic validation in more complex scenarios, and introducing quantitative evaluation based on representational coverage, model reuse rate, and model change-impact scope.

References

[1] O. Bayley, E. Savino, A. Slattery, T. Noël, Autonomous chemistry: Navigating self-driving labs in chemical and material sciences, Matter 7 (7) (2024) 2382–2398.

[2] S. D. Rihm, J. Bai, A. Kondinski, S. Mosbach, J. Akroyd, M. Kraft, Transforming research laboratories with connected digital twins, Nexus 1 (1) (2024).

[3] W. Zhang, L. Hao, V. Lai, R. Corkery, J. Jessiman, J. Zhang, J. Liu, Y. Sato, M. Politi, M. E. Reish, et al., Ivoryos: an interoperable web interface for orchestrating python-based self-driving laboratories, Nature Communications 16 (1) (2025) 5182.

[4] F. M. Mione, L. Kaspersetz, M. F. Luna, J. Aizpuru, R. Scholz, M. Borisyak, A. Kemmer, M. T. Schermeyer, E. C. Martinez, P. Neubauer, et al., A workflow management system for reproducible and interoperable high-throughput self-driving experiments, Computers & Chemical Engineering 187 (2024) 108720.

[5] J. Bai, S. Mosbach, C. J. Taylor, D. Karan, K. F. Lee, S. D. Rihm, J. Akroyd, A. A. Lapkin, M. Kraft, A dynamic knowledge graph approach to distributed self-driving laboratories, Nature Communications 15 (1) (2024) 462.

[6] J. Michael, L. Cleophas, S. Zschaler, T. Clark, B. Combemale, T. Godfrey, D. E. Khelladi, V. Kulkarni, D. Lehner, B. Rumpe, et al., Model-driven engineering for digital twins: opportunities and challenges, Systems Engineering 28 (5) (2025) 659–670.

[7] D. Lehner, J. Zhang, J. Pfeiffer, S. Sint, Model-driven engineering for digital twins: a systematic mapping study: Model-driven engineering for digital twins: a systematic mapping study, Software and Systems Modeling (2025).

[8] S. D. Rihm, Y. R. Tan, W. Ang, M. Hofmeister, X. Deng, M. T. Laksana, H. Y. Quek, J. Bai, L. Pascazio, S. C. Siong, et al., The digital lab manager: Automating research support, SLAS technology 29 (3) (2024) 100135.

[9] G. Tom, S. P. Schmid, S. G. Baird, Y. Cao, K. Darvish, H. Hao, S. Lo, S. Pablo-García, E. M. Rajaonson, M. Skreta, et al., Self-driving laboratories for chemistry and materials science, Chemical Reviews 124 (16) (2024) 9633–9732.

[10] M. Sim, M. G. Vakili, F. Strieth-Kalthoff, H. Hao, R. J. Hickman, S. Miret, S. Pablo-García, A. Aspuru-Guzik, Chemos 2.0: An orchestration architecture for chemical self-driving laboratories, Matter 7 (9) (2024) 2959–2977.

[11] Y. Fei, B. Rendy, R. Kumar, O. Dartsi, H. P. Sahasrabuddhe, M. J. McDermott, Z. Wang, N. J. Szymanski, L. N. Walters, D. Milsted, et al., Alabos: a python-based reconfigurable workflow management framework for autonomous laboratories, Digital Discovery 3 (11) (2024) 2275–2288.

[12] H. Lee, H. J. Yoo, H. S. Jang, B. Park, Y. J. Park, S. S. Han, Toward self-driving laboratory 2.0 for chemistry and materials discovery, Materials Horizons (2026).

[13] P. Salazar-Villacis, B. Benyahia, The adept framework for assessing autonomous laboratory robotics, Communications Chemistry 9 (1) (2026) 99.

[14] L. Kaspersetz, B. Englert, F. Krah, E. C. Martinez, P. Neubauer, M. N. C. Bournazou, Management of experimental workflows in robotic cultivation platforms, SLAS technology 29 (6) (2024) 100214.

[15] H. Hysmith, E. Foadian, S. P. Padhy, S. V. Kalinin, R. G. Moore, O. S. Ovchinnikova, M. Ahmadi, The future of self-driving laboratories: from human in the loop interactive ai to gamification, Digital Discovery 3 (4) (2024) 621–636.

[16] S. Lo, S. G. Baird, J. Schrier, B. Blaiszik, N. Carson, I. Foster, A. Aguilar-Granda, S. V. Kalinin, B. Maruyama, M. Politi, et al., Review of low-cost self-driving laboratories in chemistry and materials science: the “frugal twin” concept, Digital Discovery 3 (5) (2024) 842–868.

[17] S. K. Steensen, P. W. Butler, J. R. Pedersen, N. Fisker-Bødker, T. Vegge, J. H. Chang, I. E. Castelli, The necessity of dynamic workflow managers for advancing self-driving labs and optimizers, Advanced Intelligent Discovery (2026) 202500067.

[18] H. Zhang, Y. Li, S. Zhang, L. Song, F. Tao, Artificial intelligence-enhanced digital twin systems engineering towards the industrial metaverse in the era of industry 5.0, Chinese Journal of Mechanical Engineering 38 (1) (2025) 40.